

Evaluación del pre entrenamiento en redes convolucionales profundas con autoencoders

Esequiel Armeria

*Facultad de Matemática, Astronomía, Física y Computación,
Universidad Nacional de Córdoba, Ciudad Universitaria, 5000 Córdoba, Argentina*
(Dated: 13 de diciembre de 2025)

I. ABSTRACT

En este trabajo se analiza el impacto del preentrenamiento de un autoencoder convolucional a la hora de entrenar un clasificador para el conjunto Fashion-MNIST. Se compararon distintos escenarios: entrenamiento conjunto del encoder y el clasificador (fine-tuning completo) y entrenamiento exclusivo del clasificador usando el encoder congelado, tanto preentrenado como sin entrenar. Los resultados mostraron que el preentrenamiento no aporta mejoras significativas cuando el encoder se ajusta completamente, obteniéndose desempeños similares a los de un modelo entrenado desde cero. Sin embargo, cuando el encoder permanece congelado, el preentrenamiento sí produce una mejora consistente, proporcionando una representación latente más estable y adecuada para la clasificación. Además, se observó que un encoder congelado sin entrenar puede alcanzar un desempeño razonable, funcionando de manera similar a un extractor de características aleatorias. En conjunto, los experimentos indicaron que el preentrenamiento resulta beneficioso principalmente cuando el encoder se utiliza como extractor fijo de características.

II. INTRODUCCIÓN

Los autoencoders convolucionales constituyen una herramienta ampliamente utilizada para el aprendizaje no supervisado de representaciones compactas y robustas de imágenes. Su aplicación resulta especialmente adecuada en conjuntos de datos donde se desea extraer características espaciales relevantes sin recurrir inicialmente a etiquetas[1]. Un conjunto de datos que se ha convertido en un estándar moderno para la evaluación de arquitecturas de visión por computadora, es el Fashion MNIST (Modified National Institute of Standards and Technology)[2], ya que al proveer un reemplazo más desafiante para el clásico MNIST sin dejar de ser computacionalmente accesible. El uso de autoencoders convolucionales permite comprimir la información clave sin perder la forma o la ubicación de los elementos mediante convoluciones, max-pooling y capas lineales [1], [3].

El objetivo principal de este trabajo fue evaluar la efectividad del pre-entrenamiento mediante un módulo codificador aprendido de forma no supervisada, y su impacto en el rendimiento de un clasificador convolucional profun-

do. Este enfoque está fundamentado en estudios clásicos que demostraron que los autoencoders pueden actuar como mecanismos de inicialización que aceleran y estabilizan el aprendizaje supervisado [1].

III. TEORÍA

A. AutoEncoder

El autoencoder propuesto sigue la estructura típica encoder-decoder basada en convoluciones, max-pooling y convoluciones transpuestas. El encoder reduce progresivamente la resolución espacial de la entrada mientras incrementa la profundidad de los mapas de características, lo que permite una representación latente comprimida. El decoder realiza la operación inversa mediante upsampling aprendido con ConvTranspose2d (convolución transpuesta) [4]. La función de pérdida utilizada es el error cuadrático medio (MSE)

1. Arquitectura del Encoder

El codificador está compuesto por dos bloques convolucionales sucesivos diseñados para extraer características jerárquicas de la imagen de entrada de 28×28 píxeles.

El primer bloque aplica una convolución 2D con 16 filtros de tamaño 3×3 y padding igual a 1, seguida de una activación ReLU y una operación de max-pooling 2×2 , que reduce la resolución espacial a 14×14 mientras se preservan las características más relevantes. Para favorecer la regularización, se incorpora una capa Dropout con probabilidad 0,2. La elección del valor de max-pooling es debido a LeCun en base a su LeNet-5 [5] y de nuestro kernel, en las investigaciones realizadas por Simonyan y Zisserman en 2014, donde VGGNet estableció el uso sistemático de kernels 3×3 como práctica óptima [6]

El segundo bloque repite el esquema con una convolución 3×3 (padding 1) que aumenta la profundidad a 32 canales y nuevamente un max-pooling 2×2 , reduciendo la resolución a 7×7 .

Finalmente, los mapas de características resultantes se aplanan y se proyectan mediante una capa completamente conectada en un vector de dimensión 28×28 , representando la codificación comprimida del dato. Se aplica una

activación ReLU[7] y Dropout adicional para evitar sobreajuste.

2. Arquitectura del Decoder

El decodificador invierte el proceso de compresión realizado por el codificador, reconstruyendo la imagen original a partir del vector latente.

Primero, una capa totalmente conectada expande la representación comprimida desde la dimensión de entrada hasta un tensor de $32 \times 7 \times 7$, restaurando la estructura espacial mínima que tenía la salida final del codificador. Esta salida se reorganiza mediante una operación de unflatten para recuperar su forma tridimensional.

A continuación, el decodificador utiliza dos convoluciones transpuestas con $\text{stride} = 2$, cada una responsable de realizar un upsampling aprendido. La primera incrementa la resolución de 7×7 a 14×14 mientras reduce la profundidad de 32 a 16 canales, acompañada de una activación ReLU que introduce no linealidad. La segunda convolución transpuesta restaura la resolución total de la imagen de 14×14 a 28×28 y reduce la cantidad de canales de 16 a 1, produciendo así una reconstrucción final con el mismo tamaño que la entrada original.

Para garantizar que los valores de la reconstrucción permanezcan en el rango $[0,1]$, como habían estado originalmente normalizados, la etapa final aplica una función Sigmoid.

B. Clasificador Convolutivo

1. Arquitectura del Clasificador

Clasificador convolutivo basado en el codificador preentrenado. El clasificador se construye utilizando como entrada la representación latente generada por el codificador del autoencoder previamente entrenado. Este codificador permanece congelado durante la fase de aprendizaje supervisado, de modo que actúa como un extractor de características fijo y robusto.

La arquitectura del clasificador consiste en una red totalmente conectada de cuatro capas. La primera capa lineal recibe el vector producido por el codificador y lo proyecta a un espacio intermedio de 512 unidades. Esta transformación es seguida por una activación ReLU y una capa Dropout con probabilidad 0,2, utilizada como mecanismo de regularización para reducir el sobreajuste.

La segunda y tercera capa lineal reducen la dimensionalidad del espacio de representación de 512 a 256 y de 256 a 128 unidades respectivamente, nuevamente acompañadas de una activación ReLU y otro bloque de Dropout con probabilidad 0,2. Estas dos capas ocultas permiten que el modelo aprenda combinaciones no lineales

de las características extraídas por el codificador, incrementando la capacidad discriminativa del clasificador.

Finalmente, una cuarta capa lineal proyecta las 182 unidades finales a un vector de 10 salidas, correspondiente a las clases del conjunto FashionMNIST. Esta capa constituye la etapa final del módulo clasificatorio y alimenta directamente la función de pérdida de entropía cruzada [8]

IV. METODOLOGIA

La metodología se organizó en tres etapas complementarias destinadas a estudiar el comportamiento del autoencoder y del clasificador en múltiples configuraciones. A lo largo del desarrollo se realizaron varios experimentos exploratorios, pero aquí se presentan únicamente los más relevantes para el análisis final.

En la primera etapa se realizó una exploración sistemática de los hiperparámetros del autoencoder. Se variaron la tasa de aprendizaje en los valores 10^{-1} , 10^{-3} , 10^{-4} y 10^{-5} , el tamaño del lote (20, 64, 100 y 200) para evaluar la estabilidad del gradiente y la velocidad de convergencia, estos experimentos se realizaron por 30 épocas. Finalmente, se analizaron distintas cantidades de épocas para determinar el punto de saturación del aprendizaje. Esta etapa permitió seleccionar configuraciones estables para el resto de los experimentos.

En la segunda etapa se entrenó un clasificador alimentado por el encoder. Se evaluaron distintas arquitecturas modificando la profundidad y la cantidad de neuronas por capa, así como el número de épocas, con el objetivo de detectar fenómenos de sobreajuste y estimar la complejidad adecuada para la tarea.

En la tercera etapa se analizó la efectividad del preentrenamiento del encoder mediante cuatro configuraciones: encoder preentrenado y congelado, encoder sin entrenar y congelado, entrenamiento conjunto desde cero y entrenamiento conjunto con encoder previamente entrenado. Para todas las configuraciones se registraron curvas de pérdida y exactitud, y se generó una matriz de confusión para el experimento más representativo. Este diseño experimental permitió aislar de manera clara el rol del preentrenamiento en la generalización del clasificador.

V. RESULTADOS

A. Búsqueda de mejores hiperparámetros del AutoEncoder

Además de los siguientes experimentos, se observó que el mejor valor de dropout encontrado es de 0,2 con una pérdida de 0,0068

1. Optimización de la tasa de aprendizaje

En la Figura 1 se observa que a medida que la tasa de aprendizaje se va reduciendo, la pérdida inicial disminuye, pero son necesarias mas épocas para converger. En la tabla I encontramos los valores finales de nuestros entrenamiento y prueba para cada tasa implicada en el experimento. En el cual se observo que la mejor encontrada es la de 10^{-4} con un 0,00671 de pérdida en nuestro set de prueba. La tasa 10^{-1} alcanzo un 0,52626 de pérdida, la 10^{-3} un 0,00703 de pérdida, y finalmente la 10^{-5} un 0,01219 de pérdida.

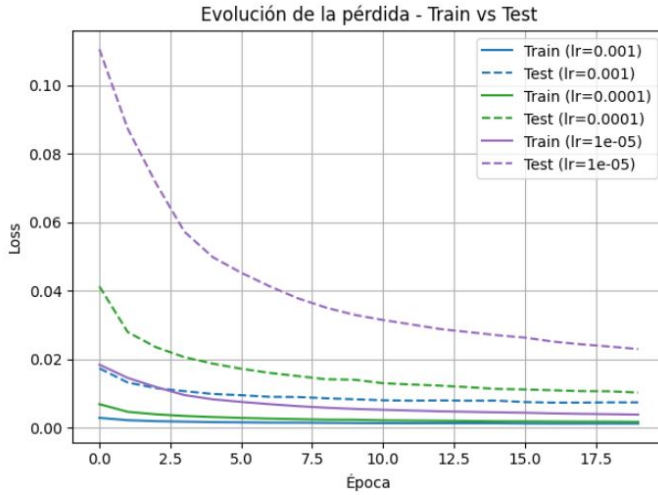


Figura 1: Curvas de pérdida de entrenamiento y validación para diferentes tasas de aprendizaje. Las líneas continuas representan el entrenamiento y las punteadas la prueba

Tabla I: Pérdida en entrenamiento y prueba para cada tasa de aprendizaje

Tasa aprendizaje	Train Loss	Test Loss
0.1	0.08771	0.52626
0.001	0.00117	0.00703
0.0001	0.00111	0.00671
0.00001	0.00203	0.01219

2. Optimización del tamaño de lote

En este experimento, veremos distinta cantidad de tamaños de lote. Se observó que, a medida que aumenta el tamaño del lote, la convergencia del autoencoder se vuelve más lenta. Los lotes grandes producen gradientes más estables pero con actualizaciones menos frecuentes, lo que reduce la variabilidad estocástica que suele favorecer el aprendizaje. En contraste, los lotes pequeños ge-

neran actualizaciones más rápidas y ruidosas, acelerando la disminución de la pérdida de reconstrucción. En este experimento, un tamaño de lote de 20 ofrece un equilibrio adecuado entre velocidad de entrenamiento y calidad de reconstrucción. (Tabla II).

Tabla II: Pérdida en entrenamiento y prueba para cada tamaño de lote

Tamaño del batch	Train Loss	Test Loss
20	0.00116	0.00701
64	0.00135	0.00809
100	0.00144	0.00869
200	0.00158	0.00953

3. Búsqueda de la mejor cantidad de épocas

En este experimento, veremos cual es la cantidad de épocas que nos dan un mejor rendimiento. La Figura 2 muestra la evolución de la pérdida de reconstrucción del autoencoder. La pérdida en entrenamiento descien- de rápidamente hasta valores muy bajos, indicando una convergencia estable del modelo, mientras que en el conjunto de prueba se observa un comportamiento similar con una meseta cercana a la curva de entrenamiento. Estos resultados reflejan una buena capacidad de reconstrucción y una generalización adecuada del autoencoder. En términos prácticos, se observa que unas 20 épocas resultan suficientes para entrenar el modelo, ya que a partir de ese punto la pérdida comienza a mostrar variaciones crecientes que no contribuyen a mejorar el rendimiento.

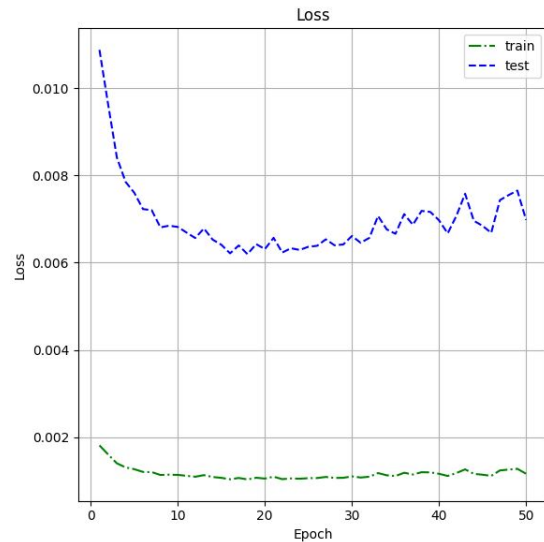


Figura 2: Comportamiento de la función de pérdida durante el entrenamiento y prueba

B. Búsqueda de mejores hiperparametros de la red clasificadora convolucional

En relacion a los hiperparametros no mostrados en los experimentos siguientes, se encontró que la tasa de aprendizaje de 0,0001 fue la que mejor precisión obtuvo, un 0,9017. Además, el tamaño del lote con mejor métrica fue el 20 con una precisión de 0,904

1. Búsqueda de la mejor cantidad de capas ocultas

En este experimento probamos distintas arquitecturas variando la cantidad de capas intermedias. Los resultados experimentales (Tabla III) muestran que la profundidad adicional no garantiza una mejora sistemática. De hecho, el modelo con ninguna capa oculta obtuvo una muy buena exactitud de 0.9056 en test, superando levemente a la arquitectura más profundas probada. No se encontró un sobre ajuste severo en ninguno de los tres modelos probados.

Tabla III: Pérdida en entrenamiento y prueba para cada cantidad de capas intermedias

Capas	Neuronas por capa	Train Accuracy	Test Accuracy
0	512→10	0.9509	0.9056
1	512→256→10	0.9663	0.9066
2	512→256→128→10	0.9610	0.9034

2. Optimización de la cantidad de neuronas por capa

En este experimento disminuirémos la cantidad de neuronas en las capas intermedias (*hidden layers*).

A partir de los valores reportados en la Tabla IV se observa una tendencia decreciente en la exactitud de prueba a medida que se reduce la cantidad de neuronas en las capas densas. Las arquitecturas con menor dimensionalidad interna muestran valores de test precisión ligeramente inferiores, lo que indica que el tamaño de las capas tiene un impacto observable en el desempeño del clasificador.

Tabla IV: Pérdida en entrenamiento y prueba para cada cantidad de neuronas en las capas intermedias

Neuronas por capa	Train Accuracy	Test Accuracy
512→256→128→10	0.9610	0.9034
256→128→64→10	0.9267	0.8988
128→64→32→10	0.9264	0.8971

3. Búsqueda de la mejor cantidad de épocas

En este experimento, veremos cual es la cantidad de épocas que nos dan un mejor rendimiento. Como se observa en la Figura 3, la pérdida de entrenamiento continúa disminuyendo a lo largo de todas las épocas, mientras que la pérdida de prueba deja de mejorar de manera apreciable después de la época 35–40. Este comportamiento es indicativo de un comienzo de sobreajuste, ya que el modelo sigue ajustándose al conjunto de entrenamiento sin obtener mejoras adicionales en el conjunto de prueba.

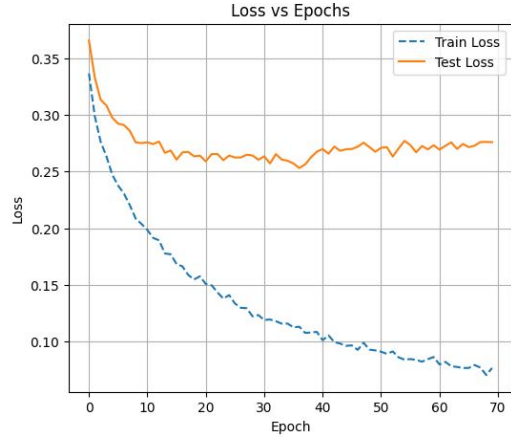


Figura 3: Comportamiento de la función de pérdida durante el entrenamiento y prueba

C. Evaluación de la efectividad del pre-entrenamiento

En los experimentos donde se entrenan conjuntamente el clasificador y el autoencoder (Experimento 1 y Experimento 2), el preentrenamiento no mostró un beneficio claro: las precisiones de prueba son muy similares (0.9254 vs. 0.9227). En cambio, cuando el encoder se mantiene congelado (Experimento 0 y Experimento 3), el preentrenamiento sí resulta útil y mejora el desempeño. Las curvas de la Figura 4 también reflejan estas diferencias. Los modelos con encoder preentrenado presentan un inicio de entrenamiento más estable y una menor brecha entre entrenamiento y prueba cuando el encoder está congelado. En los experimentos con fine-tuning completo aparece sobreajuste más temprano (época 20), mientras que el modelo con encoder congelado mantiene un comportamiento más regular. La matriz de confusión para el experimento 3 (Figura 5) muestra que los errores se concentran principalmente en prendas superiores similares, especialmente Shirt, que se confunde con T-shirt/top, Pullover y Coat. En contraste, clases con formas más distinguibles, como Trouser, Sneaker y Ankle boot, se clasifican correctamente en la mayoría de los casos.

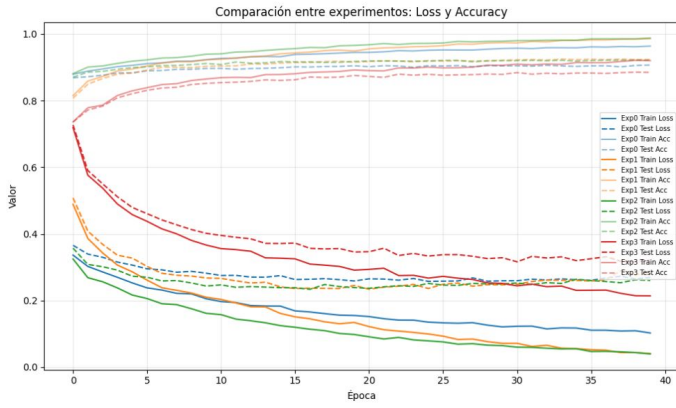


Figura 4: Comportamiento de la perdida y precision durante el entrenamiento y prueba en todos los experimentos finales

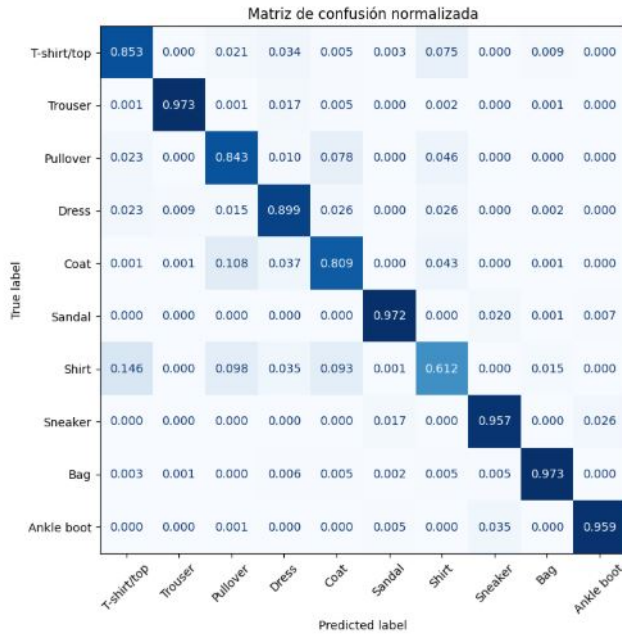


Figura 5: Matriz de confusion del clasificador en el experimento 3

Tabla V: Precision de entrenamiento y prueba cada experimento

Experimento	Pre entrenado	Train Accuracy	Test Accuracy
0	Si	0.9629	0.9069
1	No	0.9875	0.9254
2	Si	0.9879	0.9227
3	No	0.9195	0.8898

VI. CONCLUSION

Los resultados mostraron que el preentrenamiento del autoencoder no garantiza una mejora cuando el encoder se ajusta completamente durante el entrenamiento del clasificador. En los experimentos con fine-tuning completo, las precisiones finales con y sin preentrenamiento fueron muy similares, lo que indica que el clasificador es capaz de aprender representaciones útiles incluso desde pesos iniciales no especializados.

Sin embargo, cuando el encoder permanece congelado, la situación cambia de manera significativa. En este caso, el encoder preentrenado sí aporta una mejora clara en el desempeño, proporcionando una representación latente más estable y más adecuada para la tarea de clasificación. Esto evidencia que el autoencoder es útil como generador de características únicamente cuando dichas características se conservan sin modificación durante el entrenamiento.

Un resultado destacado es que incluso un encoder congelado sin preentrenar puede alcanzar un desempeño razonable, lo que sugiere que la combinación “encoder fijo + capas densas entrenables” puede funcionar como un modelo lineal sobre características aleatorias (similar a random convolutional features). Aun así, el encoder preentrenado supera sistemáticamente a su contraparte no entrenada cuando ambos están congelados, mostrando que el aprendizaje previo sí aporta estructura aprovechable.

En conjunto, los experimentos indicaron que el preentrenamiento del autoencoder es beneficioso principalmente cuando el encoder se usa como extractor fijo de características, mientras que su utilidad disminuye cuando se permite el ajuste completo de todos los parámetros del modelo.

-
- [1] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 2006. doi: 10.1126/science.1127647.
 - [2] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” *arXiv preprint arXiv:1708.07747*, 2017. Available: <https://arxiv.org/abs/1708.07747>

- [3] J. Masci, U. Meier, D. Cireşan, and J. Schmidhuber, “Stacked convolutional auto-encoders for hierarchical feature extraction,” in *Proceedings of the International Conference on Artificial Neural Networks (ICANN)*, Springer, 2011, pp. 52–59. doi: 10.1007/978-3-642-21735-7_7.
- [4] PyTorch Foundation, “PyTorch Documentation,” *PyTorch*, 2025. Available: <https://pytorch.org/docs/stable/>.
- [5] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, “Backpropagation Applied to Handwritten Zip Code Recognition,” *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989.
- [6] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” arXiv preprint arXiv:1409.1556, 2014.
- [7] A. F. Agarap, “Deep Learning using Rectified Linear Units (ReLU),” arXiv preprint arXiv:1803.08375, 2018. Available: <https://arxiv.org/abs/1803.08375>
- [8] A. Mao, M. Mohri, and Y. Zhong, “Cross-Entropy Loss Functions: Theoretical Analysis and Applications,” arXiv preprint arXiv:2304.07288, 2023. Available: <https://arxiv.org/abs/2304.07288>